Building Transformers from Scratch

Theory and Applications

A Deep Dive into Large Language Models

Tokenization • Embeddings • Attention • Transformer Blocks

Prof. Anis Koubaa

Alfaisal University

First Edition • 2026

Building Transformers from Scratch: Theory and Applications

First Edition

About the Author

Prof. Anis Koubaa is a professor of Computer Science specializing in Robotics, AI, and Deep Learning. He has extensive experience in both academia and industry, bringing practical insights to theoretical concepts.

About This Book

This textbook provides a comprehensive guide to understanding and building Transformer models from scratch. Starting with the fundamentals of tokenization and embeddings, the book guides readers through the intricacies of self-attention mechanisms, Transformer blocks, and full model architectures.

© 2026 Anis Koubaa. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form without prior written permission.

Contact: akoubaa@alfaisal.edu

Contents

2 Tokenization for Language Modeling **4**
2.1 Introduction . . . 4
2.1.1 The LLM Pipeline . . . 4
2.2 Mathematical Formalism . . . 5
2.3 From Vocabulary to Indices . . . 6
2.3.1 The Lookup Process . . . 6
2.3.2 Handling Unknown Words (OOV) . . . 6
2.4 Why Tokenization Matters . . . 6
2.5 Evolution of GPT Tokenizers: A Comparative Analysis . . . 7
2.5.1 GPT-2 Tokenizer (50k Vocabulary) . . . 7
2.5.2 GPT-3.5 (Turbo) Tokenizer . . . 7
2.5.3 GPT-4 Tokenizer (cl100k_base) . . . 8
2.6 Unicode and Byte-Level Processing . . . 8
2.6.1 What is Unicode? . . . 8
2.6.2 Why Not Use Unicode Code Points? . . . 10
2.6.3 Why Not Use Raw UTF-8 Bytes? . . . 10
2.7 Tokenization Paradigms . . . 10
2.7.1 1. Character-Level . . . 10
2.7.2 2. Word-Level . . . 10
2.7.3 3. Subword-Level (The Standard) . . . 10
2.8 Algorithm: Byte Pair Encoding (BPE) . . . 11
2.8.1 How It Works . . . 11
2.9 SentencePiece . . . 11
2.10 Special Tokens . . . 12
2.11 Tokenization Performance Metrics . . . 12
2.11.1 Intrinsic Metrics . . . 12
2.11.2 Extrinsic Metrics . . . 14
2.12 Conclusion . . . 14

2

Tokenization for Language Modeling

2.1 Introduction

Imagine trying to teach a computer to speak English. You can't just feed it raw audio waveforms or entire paragraphs at once; it's too complex. Instead, you break the language down into manageable **units of meaning**—words, syllables, or sounds.

Tokenization is the process of splitting text into these meaningful units, called **tokens**, and assigning each unit a unique linguistic identity. It is crucial to distinguish between the two key steps in this process:

1. **Tokenization (Segmentation)**: The linguistic act of breaking a string into pieces (tokens). Ideally, these pieces capture the **morphological structure** of the language.
   - Example: The word "unhappiness" is split into ‘["un", "happi", "ness"]‘.
   - This captures the prefix ‘un-‘ (negation), the root ‘happi‘ (emotion), and the suffix ‘-ness‘ (state of being). The model learns these components individually, allowing it to understand new words like "unfriendly" or "sadness" by recombining known parts.
2. **Numerical Encoding (Indexing)**: The computational act of mapping each unique token to a specific integer ID from a fixed vocabulary.
   - Example: ‘["un", "happi", "ness"]‘ → ‘[512, 3042, 109]‘.
   - The model takes these integers as input, not the text itself.

Before we dive into the complex architectures of Transformers, we must master how they "read" text. This chapter explores how we move from raw characters to intelligent subword tokens that power models like GPT-4.

2.1.1 The LLM Pipeline

Large Language Models (LLMs) like GPT-4 and Llama do not understand text directly. They are mathematical functions that operate on numerical vectors. To bridge the gap between human language and machine representation, we use a process called **tokenization**.

The pipeline for an LLM is as follows (see Figure 2.1):

1. **Input Text**: Raw natural language (e.g., "The mouse like eating").
2. **Tokenization**: The text is broken into smaller units called **tokens** (e.g., ["The", "mouse", "like", "eating"]).
3. **Token Embeddings**: These tokens are mapped to dense numerical vectors (embeddings) combined with positional information.
4. **Transformer Block**: The core neural network processes these embeddings to understand relationships (e.g., relating "mouse" to "eating").
5. **Next Token Prediction**: The model predicts the probability of the next token (e.g., "cheese": 85%).

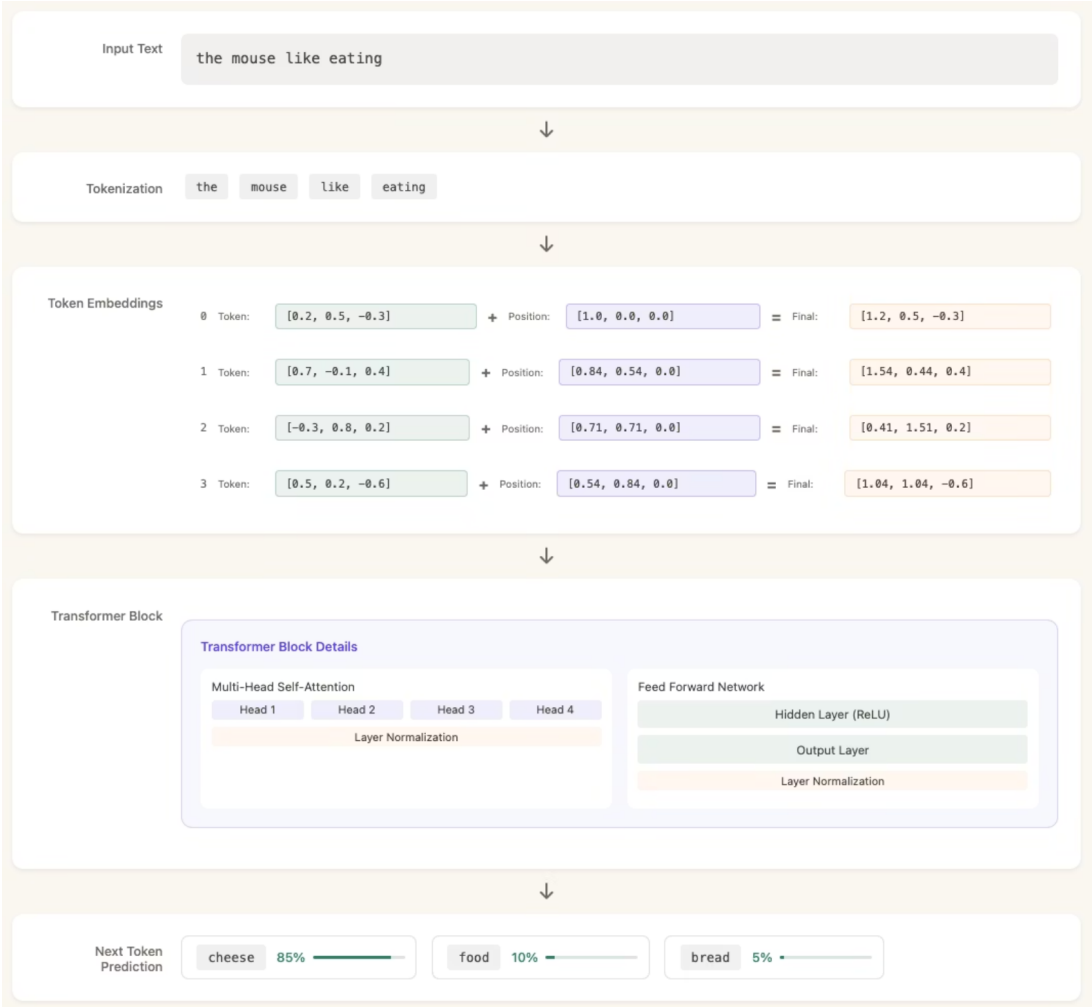

Figure 2.1: The Language Modeling Pipeline

2.2 Mathematical Formalism

Formally, we can define a tokenizer as a function Φ that maps a string sequence $\mathcal{S}$ from a character set Σ (e.g., Unicode) to a sequence of integers $\mathcal{T} \in \mathbb{Z}^N$.

$$\Phi : \Sigma^* \to \{0, 1, \ldots, |\mathcal{V}| - 1\}^N$$

Where:

- Σ^* represents the set of all possible strings.
- $\mathcal{V}$ is the **Vocabulary**, a finite set of discrete tokens.
- N is the length of the tokenized sequence.

The goal of an optimal tokenizer is often to minimize the sequence length N while keeping the vocabulary size $|\mathcal{V}|$ within computationally feasible limits (e.g., $< 100,000$).

Key Insight:

The "Goldilocks" Trade-off Tokenization is a balancing act.

- **Too Granular (Characters)**: Vocabulary is small ($\sim$100), but sequences are huge (N is large). This makes training slow and context limited.

- **Too Coarse (Words)**: Sequences are short (N is small), but vocabulary is massive ($\sim$1M+). This leads to sparse embeddings and OOV issues.
- **Just Right (Subwords)**: Algorithms like BPE find the sweet spot—keeping common words whole while breaking rare ones.

2.3 From Vocabulary to Indices

Tokenization relies on a pre-defined **Vocabulary** ($\mathcal{V}$), which is a list of all unique tokens the model knows.

2.3.1 The Lookup Process

1. **Vocabulary**: A set of tokens like ‘["hello", "world", "!"]‘.
2. **Indexing**: Each token is assigned a unique integer ID.
   - "hello" $\rightarrow$ 0
   - "world" $\rightarrow$ 1
   - "!" $\rightarrow$ 2
3. **Encoding**: The input sequence is converted to these IDs.

$$\text{"hello world!"} \rightarrow [0, 1, 2]$$

2.3.2 Handling Unknown Words (OOV)

A major challenge is handling words that are not in the vocabulary (**Out-Of-Vocabulary** or OOV).

- **Naïve Approach**: Replace with a special ‘[UNK]‘ token. This destroys information.
- **Subword Approach**: Break the unknown word into smaller known pieces.

$$\text{"universe"} \rightarrow [\text{"uni"}, \text{"verse"}]$$

This ensures the model can process any text, provided the subwords are in the vocabulary.

2.4 Why Tokenization Matters

"The better the tokenization, the smarter the model." Poor tokenization leads to:

- **Spelling Errors**: The model fails to see the internal structure of words.
- **Arithmetic Mistakes**: If numbers are split arbitrarily (e.g., "123" $\rightarrow$ ["1", "23"]), the model struggles to do math.
- **Multilingual Inefficiency**: Languages like Korean or Arabic might be fragmented into excessive numbers of tokens, diluting semantic meaning.

Key Insight:

Why "123" should be "1", "2", "3" or "123"? For a model to learn math, consistent tokenization is key. If "100" is one token but "101" is two tokens ("10", "1"), the model struggles to generalize place value. Newer tokenizers (like GPT-4’s) try to group digits more consistently (e.g., keeping up to 3 digits together) to improve arithmetic performance.

2.5 Evolution of GPT Tokenizers: A Comparative Analysis

The evolution from GPT-2 to GPT-4 demonstrates a clear trend towards efficiency, standardization, and better handling of code and multilingual text. We analyze this progression through three generations of tokenizers.

2.5.1 GPT-2 Tokenizer (50k Vocabulary)

The GPT-2 tokenizer, based on byte-level BPE, was a pioneering approach that allowed models to process any text string without needing a massive vocabulary. It utilized a vocabulary size of approximately 50,257 tokens.

However, it had significant inefficiencies. As seen in Figure 2.2, the tokenizer often fails to keep common words or numbers as single units.

1. **Inefficient Splitting**: Common words, especially in non-English languages, were often fragmented into multiple small tokens.
2. **Whitespace Handling**: It treated spaces as part of the following word in some contexts but not others, leading to inconsistent tokenization of similar phrases.
3. **High Token Count**: In the visual example, the text requires **232 tokens**. This high count limits the amount of information that can fit into the model's context window.

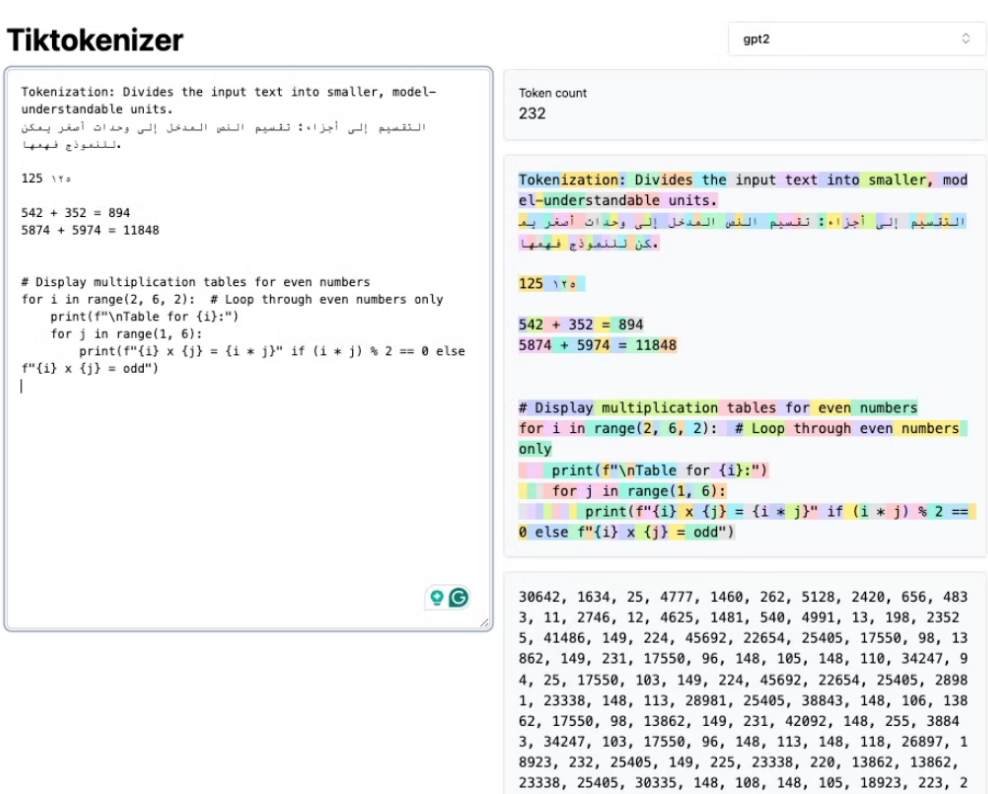

Figure 2.2: GPT-2 Tokenization: Note the inefficient splitting and high token count (232).

2.5.2 GPT-3.5 (Turbo) Tokenizer

GPT-3.5 (used in the original ChatGPT) represented an intermediate step. While it improved upon the raw efficiency of GPT-2, it still shared many underlying characteristics.

- **Vocabulary**: It largely maintained the vocabulary structure of the GPT-3 era.

- **Performance**: It achieved better compression than GPT-2 but still struggled with complex code structures and certain multilingual scripts compared to the latest generation.

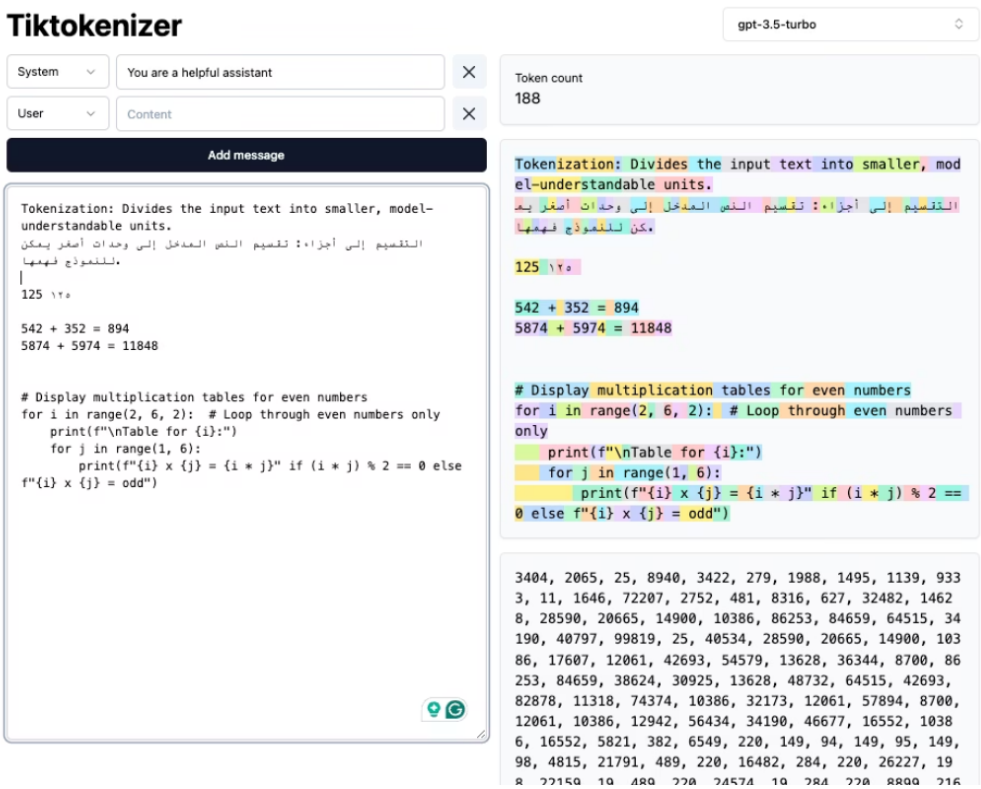

Figure 2.3: GPT-3.5 Tokenization: An improvement over GPT-2.

2.5.3 GPT-4 Tokenizer (cl100k_base)

GPT-4 introduced the **cl100k_base** tokenizer, which represents a significant leap in efficiency and semantic alignment. The vocabulary size was effectively doubled to ∼100,000 tokens.

This expansion allowed for several critical optimizations (see Figure 2.4):

1. **Digit Grouping**: Unlike GPT-2, which split numbers arbitrarily (e.g., "123" → "1", "23"), GPT-4 groups up to three digits into a single token. This consistency dramatically aids the model's ability to perform arithmetic.
2. **Code Efficiency**: The tokenizer includes specific tokens for common whitespace patterns (indentation) and programming keywords, reducing the token count for code by 30-40%.
3. **Multilingual Support**: The larger vocabulary allocates more space to common tokens in languages beyond English, making the model more "native" to global languages.
4. **Semantic Density**: By compressing text into fewer tokens (shorter sequences), the model can attend to longer contexts with the same compute budget.

2.6 Unicode and Byte-Level Processing

2.6.1 What is Unicode?

Unicode is the standard for text encoding, assigning a unique **Code Point** to over 150,000 characters (e.g., 'A' is 65, 'U+1F60A' is 128522).

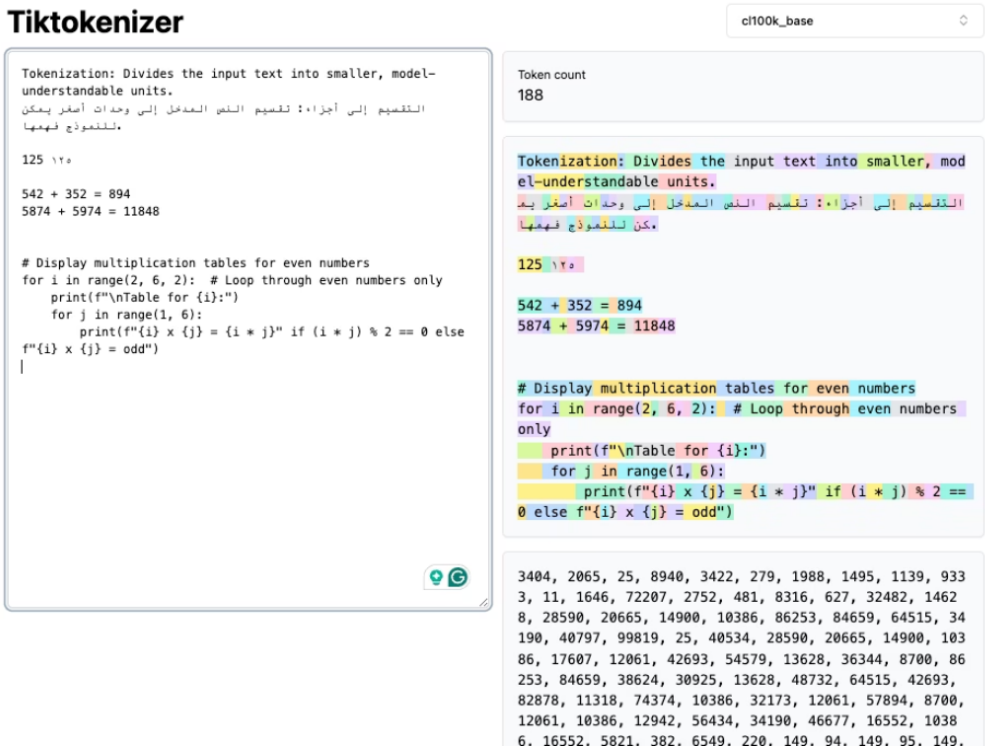

Figure 2.4: GPT-4 Tokenization: Highly compact and efficient (note the digit grouping).

Feature	GPT-2	GPT-4
Vocabulary Size	50,257	100,000
Efficiency	Lower (longer sequences)	Higher (shorter sequences)
Numbers	Splits arbitrarily	Groups digits effectively
Python Code	Spaces are separate tokens	Spaces merged efficiently

Table 2.1: Comparison of GPT-2 and GPT-4 Tokenizers

2.6.2 Why Not Use Unicode Code Points?

1. **Vocabulary Size**: 150,000+ tokens is too large for the model's output layer (embedding table becomes massive).
2. **Dynamic Standard**: New emojis and characters are added every year.
3. **No Semantics**: Code points don't capture the meaning of subwords like "un-" or "-ing".

2.6.3 Why Not Use Raw UTF-8 Bytes?

Using raw bytes (256 possibilities) solves the vocabulary size problem but introduces new issues:

1. **Sequence Length**: A single character like 'U+1F60A' becomes 4 bytes (`[240, 159, 152, 131]`). Text becomes 4-5x longer.
2. **Inefficient Attention**: Transformer complexity scales quadratically ($O(N^2)$). 4x length means 16x compute cost.

Key Insight:

The Solution We need a middle ground: **Subword Tokenization**. It uses bytes to ensure no OOV, but merges them to keep sequences short.

2.7 Tokenization Paradigms

2.7.1 1. Character-Level

- **Pros**: Tiny vocabulary (100 tokens), no OOV.
- **Cons**: Extremely long sequences, lacks semantic meaning per token.

2.7.2 2. Word-Level

- **Pros**: High semantic value, short sequences.
- **Cons**: Massive vocabulary, many OOV words, fails on rich morphology.

2.7.3 3. Subword-Level (The Standard)

Balances the two. It keeps frequent words whole ("the", "apple") and breaks rare words ("unfriendly" → "un", "friend", "ly").

Challenge: Arabic Morphology

Arabic is morphologically rich. A single word often contains a prefix, root, and suffix. For example, the word "Wakatabna" means "And we wrote". Traditional word tokenization fails here because every combination matches a unique dictionary word, leading to an explosion of vocabulary. Subword tokenization is essential for Arabic to split these components.

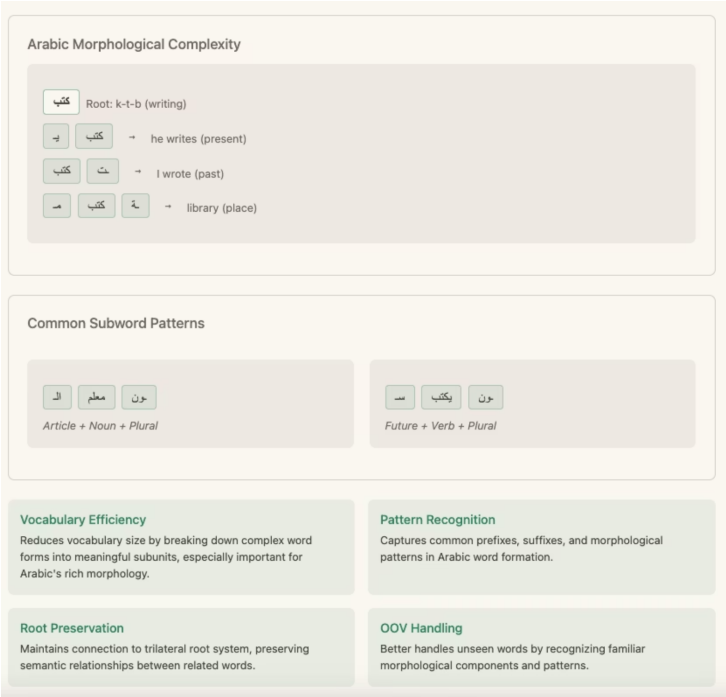

Figure 2.5: Arabic Morphology Example

2.8 Algorithm: Byte Pair Encoding (BPE)

BPE is a compression algorithm used by GPT-2, GPT-3, and GPT-4.

2.8.1 How It Works

1. **Initialize**: Start with a vocabulary of individual characters (or bytes).
2. **Count Pairs**: Find the most frequent adjacent pair of tokens in the corpus.
3. **Merge**: Combine them into a new single token. Add to vocabulary.
4. **Repeat**: Continue until the target vocabulary size is reached.

Example:

BPE Walkthrough **Corpus**: "low lower lowest"
Initial: [l, o, w, _, l, o, w, e, r, _, l, o, w, e, s, t]
Most Frequent: ('l', 'o')
Merge 1: Create token 'lo'. Corpus: [lo, w, _, lo, w, e, r, _, lo, w, e, s, t]
Most Frequent: ('lo', 'w')
Merge 2: Create token 'low'. Corpus: [low, _, low, e, r, _, low, e, s, t]

2.9 SentencePiece

SentencePiece (by Google) is used in LLaMA and Mistral.

- **Lossless**: Treats whitespace as a distinct symbol (e.g., U+2581 _).
- **Raw Input**: Does not require pre-tokenization (splitting by space) like original BPE.
- **Byte Fallback**: If a character is not in the vocabulary, it falls back to its UTF-8 byte representation, ensuring no [UNK] tokens.

2.10 Special Tokens

Special tokens add structure to the model's input.

- `<|endoftext|>`: Marks the end of a document.
- `<|im_start|>`, `<|im_end|>`: Delimit turns in a conversation (User vs. Assistant).
- `<|fim_prefix|>`: Used for code completion (Fill-In-The-Middle).

2.11 Tokenization Performance Metrics

2.11.1 Intrinsic Metrics

These measure the quality of the vocabulary itself.

1. Fertility

Average number of tokens per word. Lower is generally better (closer to 1.0 means words are kept whole).

$$\text{Fertility} = \frac{\text{Total Tokens}}{\text{Total Words}}$$

Figure 2.6: Fertility Rate Comparison

2. Single Word Token Performance

Percentage of words that are tokenized as a single unit. High values indicate a vocabulary well-suited to the domain.

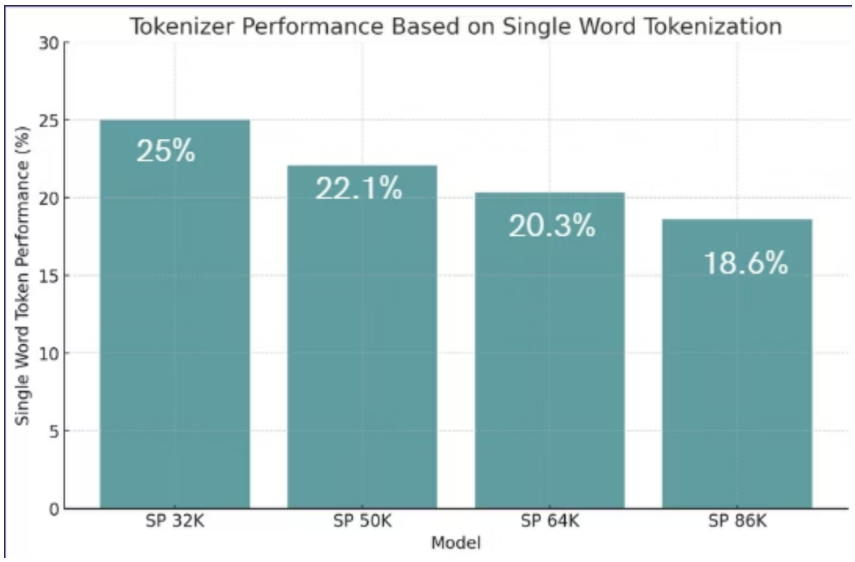

Figure 2.7: Single Word Token Performance

3. Root Token Performance

Ability to protect the root of the word. For "running", we want ["run", "ning"], not ["ru", "nning"]. Capturing the root preserves semantics.

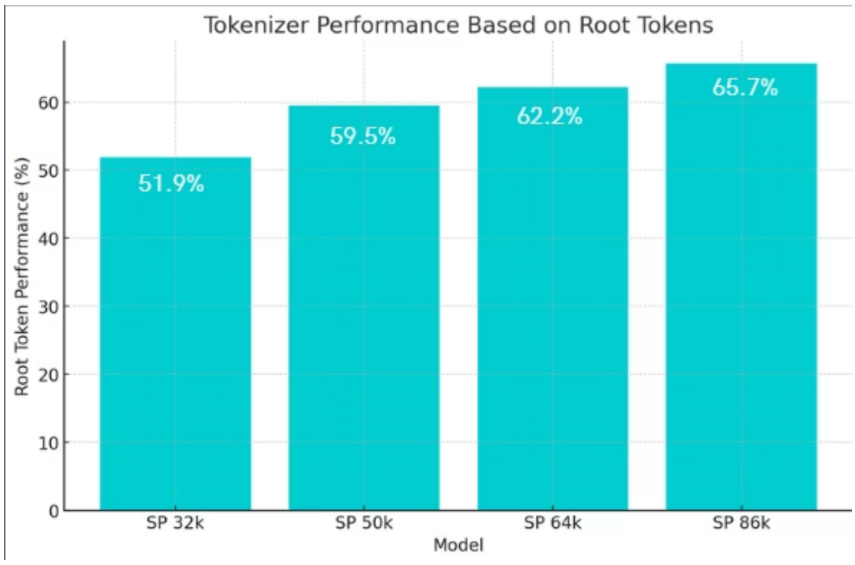

Figure 2.8: Root Token Performance

4. Suffix Precision

When a word is split, is the suffix a real linguistic suffix? (e.g., "-ing" is good, "-ng" is bad).

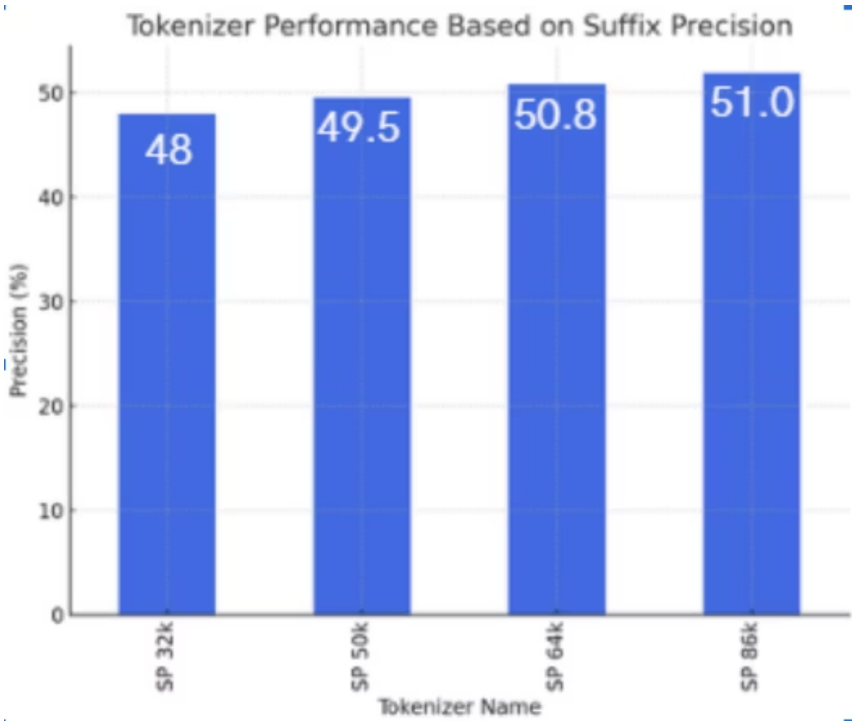

Figure 2.9: Suffix Precision

2.11.2 Extrinsic Metrics

- **Downstream Performance**: Accuracy on real tasks (translation, summarization).
- **Computational Cost**: Depends on sequence length (Fertility).

2.12 Conclusion

Tokenization is more than just string splitting; it is a critical compression step that defines how an LLM views the world. By moving from simple heuristics to data-driven algorithms like BPE and SentencePiece, we enable models to handle diverse languages, code, and complex morphology efficiently.